Smart Thermostat System

Example program: priorities, guard clauses and combined conditions

The problem

Decide what the heating/cooling unit should do from the room temperature, the target, the mode and whether anyone is home. Rules are checked top to bottom.

#	Rule	Result
1	Safety: `temp >= 40` or `temp <= 5`	ALERT
2	Mode is `"off"`	OFF
3	Comfort band: 1 degree at home, 4 when away	(sets `band`)
4	Too hot and mode is `auto` or `cool`	COOLING
5	Too cold and mode is `auto` or `heat`	HEATING
6	Anything else	IDLE

Try it

```
print(thermostat(30, 22))              # hot
print(thermostat(18, 22))              # cold
print(thermostat(22.5, 22))            # within band
print(thermostat(26, 22, "heat"))      # hot, heat-only
print(thermostat(26, 22, home=False)) # away: wide band
print(thermostat(30, 22, home=False)) # far outside
print(thermostat(45, 22))              # sensor fault
print(thermostat(20, 22, "off"))       # switched off
```

OUTPUT

```
COOLING
HEATING
IDLE
IDLE
IDLE
COOLING
ALERT: check sensor
OFF
```

Extend it: target by time of day

```
def target_for(hour, home):
    if not home:
        return 17
    if 6 <= hour < 9 or 17 <= hour < 23:
        return 22                # morning, evening
    if hour >= 23 or hour < 6:
        return 19                # night
    return 20                    # daytime
print(target_for(7, True), target_for(2, True),
      target_for(13, False))
```

OUTPUT

```
22 19 17
```

Concepts used

Concept	Where it appears
Guard clause (early `return`)	safety and OFF checks exit at once
Priority by order	safety test is written first
Combined test `or` / `and`	`temp >= 40 or temp <= 5`
Membership `in`	`mode in ("auto", "cool")`
Ternary expression	`1 if home else 4`
Default arguments	`mode="auto", home=True`
Negative threshold	`diff < -band`

The program

```
def thermostat(temp, target, mode="auto", home=True):
    # 1. safety wins over everything
    if temp >= 40 or temp <= 5:
        return "ALERT: check sensor"
    # 2. system switched off
    if mode == "off":
        return "OFF"
    # 3. wider comfort band when nobody is home
    band = 1 if home else 4
    diff = temp - target
    # 4-6. pick the action
    if diff > band and mode in ("auto", "cool"):
        return "COOLING"
    elif diff < -band and mode in ("auto", "heat"):
        return "HEATING"
    return "IDLE"
```

WATCH OUT Order is priority. If the OFF test came first, a frozen pipe or an overheating room would never raise an alert while the system is off.

Interactive version

```
temp = float(input("Room temp (C): "))
target = float(input("Target (C): "))
mode = input("Mode [auto/heat/cool/off]: ").lower()
home = input("Anyone home? (y/n): ") == "y"
print(thermostat(temp, target, mode, home))
```

OUTPUT

```
COOLING
```

Decision flow

```
temp out of safe range? ──yes──► ALERT
        | no
mode is "off"? ──────────yes──► OFF
        | no
band = 1 (home) or 4 (away)
        |
too hot  and auto/cool? ─yes─► COOLING
        | no
too cold and auto/heat? ─yes─► HEATING
        | no
        ▼
      IDLE
```

Test it with assert

```
assert thermostat(30, 22) == "COOLING"
assert thermostat(18, 22) == "HEATING"
assert thermostat(22.5, 22) == "IDLE"
assert thermostat(45, 22, "off").startswith("ALERT")
print("all tests passed")
```

OUTPUT

```
all tests passed
```

REMEMBER

- Put the most important (safety) rule first and let guard clauses `return` early: no deep nesting.
- Name magic numbers: `MAX_SAFE = 40`, `MIN_SAFE = 5`. Rules then live in one place.
- Test every branch: `assert thermostat(30, 22) == "COOLING"` takes one line.
- Add hysteresis (switch off at the target, on at target +/- band) to avoid rapid on/off cycling.
- Return a dict such as `{"action": "COOLING", "fan": "high"}` when callers need more than a string.
- Extension idea: a humidity branch, or `match mode:` once each mode has its own behaviour.